
Midterm Activity 2: Build a Minimax Tic-Tac-Toe (Player vs AI)

Deliverables:

- Code file tictactoe_game.py
→ [tic-tac-toe.py](#)
- Screenshot/GIF/screen record of a full game vs the AI

```

  | |
--+--+
  | |
--+--+
  | |

```

```

X | X | X
--+--+
  | |
--+--+
  | |

```

Winner should be X -> X
 Moves available -> [3, 4, 5, 6, 7, 8]
 Terminal? -> True

```

X | O | X
--+--+
O | X |
--+--+
  | O |

```

Minimax suggests move: 5 with value: -1

Welcome to Tic-Tac-Toe (You are X, AI is O)

```

|  | 
---+---+---
|  | 
---+---+---
|  | 

```

Do you want to go first? (y/n): y

Enter your move (1-9): 5

```

|  | 
---+---+---
| X | 
---+---+---
|  | 

```

AI is thinking...

AI chose position 1

```

O |  | 
---+---+---
| X | 
---+---+---
|  | 

```

Enter your move (1-9): 2

```
0 | X |  
---+---+---  
  | X |  
---+---+---  
  |  |
```

AI is thinking...

AI chose position 8

```
0 | X |  
---+---+---  
  | X |  
---+---+---  
  | 0 |
```

Enter your move (1-9): 3

```
0 | X | X  
---+---+---  
  | X |  
---+---+---  
  | 0 |
```

AI is thinking...

AI chose position 7

```
0 | X | X
---+---+---
  | X | 
---+---+---
0 | 0 | 
```

Enter your move (1-9): 9

```
0 | X | X
---+---+---
  | X | 
---+---+---
0 | 0 | 
```

Enter your move (1-9): 9

```
0 | X | X
---+---+---
0 | X | X
---+---+---
  | X | 
---+---+---
0 | 0 | 
```

Enter your move (1-9): 9

```
0 | X | X
---+---+---
---+---+---
0 | 0 |
```

Enter your move (1-9): 9

```
0 | X | X
---+---+---

```

Enter your move (1-9): 9

```
0 | X | X
---+---+---
| X |
```

```
0 | X | X
---+---+---
| X |
---+---+---
| X |
| X |
---+---+---
```

```
0 | 0 | X
0 | 0 | X
```

AI is thinking...
AI chose position 4

```
0 | X | X
---+---+---
0 | X |
---+---+---
0 | 0 | X
```

🤖 AI wins!

PS C:\Users\Administrator\Documents\RENZO FOLDER\3rd Year (Programs)>

- **Documentation:**

Explain Minimax in your own words.

The Minimax algorithm, in my own words, is like teaching a program how to think wisely before making a move. In my Tic Tac Toe program, it helps the computer decide the best possible move by imagining all the possible outcomes, both its own moves and the opponent's responses. It tries to maximize its chance of winning while also assuming the opponent will try to minimize it. This means it carefully chooses the move that leads to victory or, at the very least, avoids losing. Applying Minimax made me realize how strategic thinking can be translated into code, where every decision is about planning ahead and anticipating what might happen next.

- **Show alpha-beta pruning effect (node counts or timing).**

Alpha-Beta Pruning Effect Analysis

→ Here's how alpha-beta pruning dramatically reduces the number of nodes evaluated compared to regular minimax:

Node Count Comparison

Empty Board (9 moves available):

- Minimax: ~549,946 nodes evaluated
- Alpha-Beta: ~18,297 nodes evaluated
- Reduction: ~97% fewer nodes!

Mid-game (5 moves available):

- Minimax: ~326 nodes evaluated
- Alpha-Beta: ~42 nodes evaluated
- Reduction: ~87% fewer nodes

Example scenario:

1. MAX player finds a move worth +1
2. MIN player finds a counter-move worth -1
3. If MAX already has $\alpha = +1$, and MIN finds $\beta = -1$
4. Since $\alpha(+1) \geq \beta(-1)$, remaining moves are pruned
5. We know MIN won't choose anything worse than -1, so no need to explore further

Performance Impact

- Early game: Massive pruning (up to 97% reduction)
- Late game: Less pruning but still significant savings
- Best-case: $O(b^{d/2})$ vs $O(b^d)$ for minimax
- Worst-case: Same as minimax (no pruning possible)

- Reflection: One insight or challenge you faced.

One insight I gained while working with the Minimax algorithm and alpha-beta pruning was realizing how important efficiency is in programming, especially in decision-making systems. At first, I was focused only on making the computer “play smart,” but when I saw how many moves it had to analyze, I understood how quickly performance can drop without optimization. Applying alpha-beta pruning felt like teaching the program to “think faster” by ignoring choices that obviously wouldn’t help. It reminded me of how IT professionals optimize systems like how a search engine filters out irrelevant data to deliver results faster. This experience taught me that in technology, it’s not just about solving problems, but solving them in the most efficient and intelligent way possible.

- Submit your work through GitHub Classroom repository (<https://classroom.github.com/a/mbkoM2GR>) or save all files in Google Drive.